

FUNDINDO CARACTERÍSTICAS ESPACIAIS E TEMPORAIS COM PORTAS TRANSFORMER: UM MODELO HÍBRIDO DE APRENDIZADO PROFUNDO PARA DETECÇÃO DE INTRUSÕES EM SISTEMAS

Eduardo Fontes Baltazar da Silveira

Universidade Federal da Bahia, Engenharia da Computação, eduardosilveira@ufba.br

RESUMO

O presente trabalho desenvolve um modelo híbrido de aprendizado profundo para detecção de intrusões em sistemas, utilizando técnicas de fusão de características espaciais e temporais através de portas Transformer. A pesquisa utiliza o dataset UNSW-NB15, reconhecido por sua representatividade em tráfego normal e malicioso, abordando desafios de desbalanceamento de classes e sobreposição de padrões. O estudo implementa uma arquitetura que combina redes neurais convolucionais (CNN) e redes long short-term memory (LSTM) com mecanismos de atenção, alcançando resultados significativos na classificação de ataques cibernéticos.

Palavras-chave: Detecção de Intrusão. Aprendizado Profundo. Transformer. CNN-LSTM. Segurança Cibernética.

SUMÁRIO

1 INTRODUÇÃO.....	2
2 ANÁLISE DO DATASET.....	2
3 EXPERIMENTAÇÃO E TESTES.....	4
4 PIPELINE DE PRÉ-PROCESSAMENTO.....	5
5 TRANSFORMAÇÃO DE SEQUÊNCIAS.....	7

6 TRANSFORMAÇÃO DE DADOS EM IMAGENS.....	9
7 ANÁLISE TÉCNICA DO MODELO HÍBRIDO.....	10
8 TREINAMENTO.....	12
9 ANÁLISE DOS RESULTADOS.....	15
10 POSSÍVEIS MELHORIAS.....	19
11 CONCLUSÃO.....	21
REFERÊNCIAS.....	23

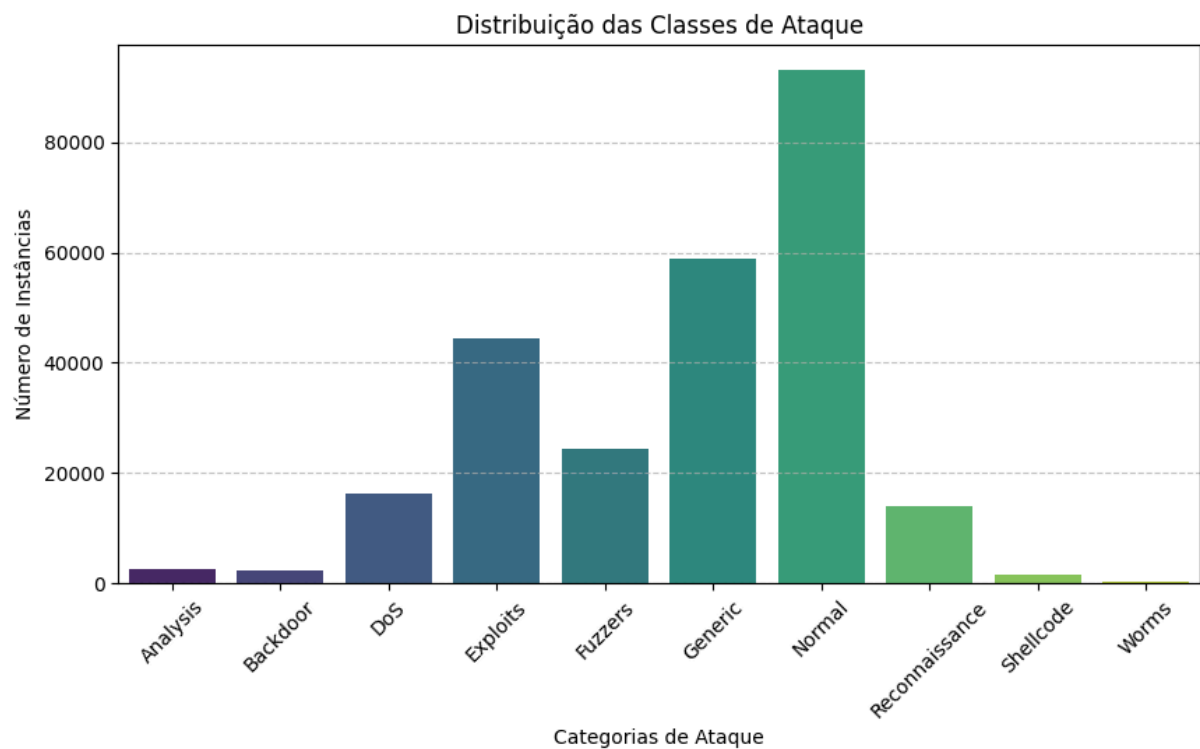
1 INTRODUÇÃO

A segurança cibernética representa uma área de crescente importância no contexto atual de redes interconectadas. Este trabalho propõe a aplicação de técnicas de aprendizado profundo para classificação de ataques cibernéticos utilizando o dataset UNSW-NB15, reconhecido por sua ampla representatividade de tráfego normal e malicioso.

2 ANÁLISE DO DATASET

O UNSW-NB15 constitui um conjunto de dados amplamente utilizado na pesquisa de sistemas de detecção de intrusões devido à sua riqueza e complexidade.

2.1 Classes do Dataset



O conjunto de dados apresenta dez classes distintas, sendo uma representativa de tráfego normal e nove de diferentes categorias de ataques, conforme descrito a seguir.

2.1.1 Tráfego Normal

Caracteriza-se como tráfego legítimo e não malicioso, apresentando desafios relacionados ao desbalanceamento devido à sua predominância no conjunto de dados.

2.1.2 Analysis

Compreende ataques focados na exploração de vulnerabilidades para coleta de informações, como varreduras de portas. Apresenta características similares a auditorias legítimas, o que pode dificultar a classificação.

2.1.3 Backdoor

Caracteriza-se pela instalação de acessos remotos não autorizados, apresentando conexões persistentes de baixa intensidade.

2.1.4 DoS (Denial of Service)

Consiste em ataques que visam sobrecarregar sistemas, caracterizados por altas taxas de pacotes em intervalos curtos.

2.1.5 Exploits

Refere-se a ataques que exploram vulnerabilidades específicas, apresentando alta variabilidade devido aos seus subgrupos.

2.1.6 Fuzzers

Caracteriza-se pelo envio de dados aleatórios para identificação de vulnerabilidades, gerando tráfego anômalo de baixa intensidade.

2.1.7 Generic

Afeta múltiplos algoritmos de criptografia, apresentando padrões regulares que facilitam sua detecção.

2.1.8 Reconnaissance

Focado na coleta de informações sobre a rede, caracteriza-se por múltiplas conexões curtas para diferentes destinos.

2.1.9 Shellcode

Consiste em código malicioso inserido em tráfego legítimo para assumir controle de sistemas.

2.1.10 Worms

Caracteriza-se por ataques automáticos com padrões regulares de propagação através das redes.

2.2 Distribuição das Features

O dataset apresenta 49 colunas descritivas do tráfego de rede, organizadas nas seguintes categorias:

- a) Features Básicas: estatísticas gerais sobre o tráfego;
- b) Features de Conteúdo: informações detalhadas sobre pacotes;
- c) Features Temporais: comportamento temporal;
- d) Features Gerais: indicadores de conexão;
- e) Features de Conexão: relacionamentos entre fluxos consecutivos;
- f) Features de Classificação: rotulagem para aprendizado supervisionado.

2.3 Desafios Associados

Os principais desafios identificados incluem:

- a) Desbalanceamento entre classes majoritárias e minoritárias;
- b) Sobreposição entre classes com comportamentos similares;
- c) Variabilidade interna em classes como "Exploits".

3 EXPERIMENTAÇÃO E RESULTADOS

3.1 Metodologias Aplicadas

Foram realizados experimentos com diferentes abordagens:

- a) Balanceamento de classes através de oversampling e undersampling;
- b) Desenvolvimento de modelos binários por classe;
- c) Tratamento de dados duplicados;
- d) Remoção de classes minoritárias;
- e) Redução de sobreposição entre classes.

3.2 Resultados Obtidos

Os experimentos demonstraram que:

- a) Técnicas de custo-sensível apresentaram melhor desempenho que oversampling;
- b) Modelos binários mostraram desempenho insatisfatório para classes minoritárias;
- c) A remoção de dados duplicados impactou negativamente a detecção de algumas classes;
- d) A exclusão de classes minoritárias resultou em aumento da precisão global.

3.3 Conclusões da Seção

A análise e experimentação com o dataset UNSW-NB15 evidenciaram que a eficácia de sistemas de detecção de intrusão depende fundamentalmente da adaptação à natureza intrínseca do tráfego de rede. Abordagens práticas, como seleção adequada de features e priorização de classes operacionalmente relevantes, demonstraram maior impacto que métodos teóricos de otimização.

4 PIPELINE DE PRÉ-PROCESSAMENTO

4.1. Fundamentos Metodológicos

A pipeline de pré-processamento foi projetada considerando os desafios específicos do dataset UNSW-NB15, com foco em:

- Lidar com desbalanceamento de classes;
- Preservar padrões complexos de ataques;
- Maximizar a capacidade de generalização do modelo.

4.2. Estratégias de Pré-processamento

4.2.1. Tratamento de Classes

Critérios de Seleção:

- Remover classes com menos de 5% de representatividade;
- Priorizar classes operacionalmente relevantes;
- Manter pelo menos 20 amostras por categoria remanescente.

Resultados:

- Reduz ruído e complexidade do modelo;
- Foca em padrões de ataque mais significativos;
- Previne overfitting em classes marginais.

4.2.2. Codificação de Features

Técnicas Aplicadas:

- Frequency encoding para variáveis categóricas;
- Preservação de informação contextual;
- Tratamento de categorias desconhecidas.

Vantagens:

- Redução dimensional;
- Evita data leakage;
- Robustez a novas categorias em produção.

4.2.3. Engenharia de Features

Categorias de Features Criadas	Tipo	Objetivo			Exemplos
Métricas de Perda	Detecção DDoS	de	Identificar perda de pacotes	de	loss_ratio, packet_error_ratio
Performance de Rede	Identificar Flooding		Analisar excessivo	tráfego	throughput, tcp_window_ratio
Assimetria de Tráfego	Detectar Ataques Sutis		Identificar incomuns	padrões	load_ratio, jitter_ratio
Padrões TCP	Identificar Scanning		Detectar TCP	anomalias	ack_error_ratio, port_conn_ratio

4.2.4. Seleção de Features

Metodologia:

- Boruta com LightGBM (LGBM);
- Seleção estatística ($\alpha=0.01$);
- Eliminação recursiva de features.

Benefícios:

- Considera interações complexas;
- Mantém features contextualmente relevantes;
- Reduz dimensionalidade preservando interpretabilidade.

4.2.5. Tratamento de Outliers e Normalização

Estratégias:

- Isolation Forest para detecção de outliers (1% remoção);
- StandardScaler para normalização;
- Preservação de distribuições originais.

4.2.6. Divisão de Dados

Abordagem:

- Estratificação hierárquica;
- Preservação da distribuição original;
- Semente aleatória fixa para reprodutibilidade.

4.3. Princípios Fundamentais

4.3.1. Pragmatismo sobre Teorização

- Priorizar adaptação prática;
- Focar na natureza intrínseca do tráfego.

4.3.2. Tolerância a Complexidade

- Aceitar sobreposições entre classes;
- Modelar variabilidade inerente.

4.3.3. Engenharia Contextual

- Features que capturem nuances de ataques;
- Transformações que preservam informação.

4.4. Conclusão Metodológica

A pipeline visa criar um modelo de detecção de intrusão que:

- Seja robusto a variações de tráfego;
- Minimize falsos positivos;
- Capture padrões sutis de ataques cibernéticos.

O foco está na qualidade da transformação, não na quantidade de dados manipulados.

5 TRANSFORMAÇÃO DE SEQUÊNCIAS

5.1. Visão Geral do Processo

A transformação de sequências é implementada através da classe `SequenceGenerator`, que herda de `tf.keras.utils.Sequence`. Essa implementação fornece um framework robusto para processamento de dados sequenciais em detecção de intrusão, com foco especial na preservação de relações temporais e contextuais.

5.2. Categorização de Features

5.2.1. Features Temporais

- Características que variam ao longo do tempo.

5.2.2. Features Estáticas

- Características que permanecem constantes durante a sequência.

5.3. Técnicas de Transformação Sequencial

5.3.1. Agrupamento por Similaridade

- Utilização de K-means para agrupar sequências similares.
- Número de clusters configurável (padrão: 10).
- Benefício: Melhoria na consistência das sequências geradas.

5.3.2. Enriquecimento de Sequências

Técnicas Principais:

- Adição de Ruído: Aumenta a robustez do modelo, previne overfitting e possui nível de ruído configurável.
- Agregação Estatística: Adiciona estatísticas globais, captura tendências gerais e facilita a identificação de padrões.
- Codificação Posicional: Preserva informação temporal, permite atenção efetiva e é baseada na arquitetura Transformer.

5.4. Geração de Sequências

5.4.1. Parâmetros de Controle

- `sequence_length`: Tamanho da sequência (default: 5).
- `min_sequence_elements`: Mínimo de elementos por sequência.
- `stride`: Passo entre sequências consecutivas.
- `batch_size`: Tamanho do lote de processamento.

5.4.2. Processo de Geração

- Preparação dos Dados: Separação de features temporais e estáticas, agrupamento por similaridade e geração de índices válidos.
- Construção de Lotes: Padding de sequências, aplicação de transformações e geração de máscaras de atenção.

5.5. Máscaras de Atenção

- Identifica elementos válidos na sequência.
- Ignora padding durante a atenção.
- Melhora eficiência do treinamento.

5.6. Configuração e Compatibilidade

5.6.1. Formato de Saída

- Formato estruturado para integração com TensorFlow.

5.6.2. Integração com TensorFlow

- Herança de `tf.keras.utils.Sequence`.
- Conversão automática para tensores.
- Suporte a treinamento distribuído.

5.7. Impacto no Modelo

Benefícios:

- Captura de padrões temporais.
- Robustez a variações.
- Eficiência computacional.

Considerações:

- Overhead de processamento.
- Consumo de memória.
- Complexidade do treinamento.

6. TRANSFORMAÇÃO DE DADOS EM IMAGENS PARA DETECÇÃO DE INTRUSÃO

6.1 Fundamentos e Arquitetura

A classe *UNSWNB15ToImage* implementa uma camada personalizada do TensorFlow que transforma dados de rede em representações visuais, permitindo a aplicação de técnicas de visão computacional para detecção de intrusão.

6.1.1 Arquitetura Base

- Herança de `tf.keras.layers.Layer`;
- Integração com processador existente;
- Configuração flexível de dimensões;
- Suporte a cache para melhor desempenho;
- Injeção controlada de ruído para aumento de dados.

6.2 Pipeline de Processamento de Features

6.2.1 Gerenciamento de Estatísticas

- Rastreamento dinâmico de ranges de features;
- Atualização online de estatísticas;
- Gerenciamento de estado de inicialização;
- Ordenação de features baseada em correlação.

6.2.2 Normalização de Features

- Normalização *min-max* robusta;
- Tratamento de divisão por zero;
- *Clipping* de valores para $[0, 1]$;
- Processamento específico para treino.

6.3 Processo de Geração de Imagens

6.3.1 Otimização de Dimensões

- Cálculo automático de dimensões;
- Garantia de potência de 2;
- Tamanho mínimo garantido (8x8).

6.3.2 Construção da Imagem

- Cálculo dinâmico de *padding*;
- Preservação da dimensão do *batch*;
- Geração de imagem em escala de cinza;
- Redimensionamento bilinear com *anti-aliasing*.

6.4 Estratégias de Aumentação de Dados

6.4.1 Transformações Geométricas

- Rotações aleatórias de 90 graus;
- Deslocamentos espaciais controlados;
- Aumentações apenas durante treino.

6.4.2 Injeção de Ruído

A classe implementa injeção controlada de ruído durante o treino:

- Ruído gaussiano com nível configurável;
- Aplicação de ruído no espaço de features;
- Aprimoramento específico para fase de treino.

6.5 Pipeline de Dados Otimizado

6.5.1 Fluxo de Processamento

- Tratamento flexível de entrada;
- *Batching* automático;
- Otimização de processamento paralelo;
- Cache opcional de resultados;
- *Prefetching* para melhor performance.

6.6 Benefícios da Abordagem

6.6.1 Capacidade de Aprendizado Aprimorada

- Compatibilidade com *CNNs*;
- Preservação de padrões espaciais;
- Relações robustas entre features.

6.6.2 Otimização de Performance

- Processamento eficiente em *batch*;
- Otimização de grafo TensorFlow;
- Suporte a execução paralela.

6.6.3 Análise e *Debugging*

- Suporte a visualização direta;
- Interpretação de padrões;
- Ferramentas integradas de visualização.

7. ANÁLISE TÉCNICA DO MODELO HÍBRIDO CNN-LSTM COM FUSÃO TRANSFORMADORA

7.1. Base Científica

Singh & Jang-Jaccard (2022) - Abordagem Convolutacional Multi-Escala

Contribuição Chave: Demonstração da eficácia de convoluções dilatadas para captura de padrões em diferentes escalas espaciais.

Aplicação no Modelo:

- Implementação de blocos convolucionais paralelos com taxas de dilatação (1, 2, 3);
- Uso de *depthwise convolutions* para eficiência computacional;
- Estratégia de fusão hierárquica de características multi-escala.

Melhoria Implementada:

- Sistema de SE Blocks adaptativos com compressão dinâmica mínima de 4 canais.

Imrana et al. (2021) - Arquitetura CNN-RNN para Dados Temporais

Contribuição Chave: Integração eficiente de redes convolucionais e recorrentes para dados multimodais.

Aplicação no Modelo:

- *Pipeline* paralelo de processamento espacial-temporal;
- Mecanismo de sincronização de *features* CNN-LSTM;
- Sistema de regularização cruzada entre modalidades.

Inovação Adicional:

- Mecanismo *TransformerFusion* substituindo concatenação direta.

Acharya et al. (2023) - Modelo Híbrido CNN-LSTM

Contribuição Chave: Validação empírica de arquiteturas bidirecionais para sequências temporais complexas.

Aplicação no Modelo:

- *Stack* de LSTMs bidirecionais com normalização por camada;
- Sistema de *pooling* temporal duplo (média + máximo);
- Regularização espacial progressiva.

Otimização Implementada:

- Bloco SE temporal para recalibração dinâmica de sequências.

7.2. Arquitetura Detalhada e Escolhas de Design

7.2.1 Componentes Principais e Fluxo de Dados

1. Arquitetura CNN Multi-Escala

Contribuição Principal: Implementação de convoluções dilatadas para captura eficiente de padrões em diferentes escalas espaciais.

Aplicação no Modelo:

- Blocos convolucionais paralelos com diferentes taxas de dilatação;
- Blocos SE (*Squeeze-and-Excitation*) adaptativos;

- Convoluções separáveis em profundidade;
 - Estratégia de fusão hierárquica.
 - O módulo CNN é responsável pela extração de características espaciais.
2. **Módulo LSTM Bidirecional**
Objetivo: Captura de dependências temporais de longo e curto prazo.
Aplicação no Modelo:
- Processamento bidirecional de sequências temporais;
 - Normalização por camada para estabilidade;
 - *Dropout* espacial para regularização;
 - Bloco SE temporal para recalibração dinâmica das sequências.
3. **Mecanismo de Fusão TransformerFusion**
Objetivo: Melhorar a integração entre características espaciais (CNN) e temporais (LSTM).
Aplicação no Modelo:
- *LayerNormalization* para CNN e LSTM antes da fusão;
 - Uso de *MultiHeadAttention* para realçar relações entre diferentes fontes de informação;
 - Implementação de portão dinâmico (*Dense(sigmoid)*) para ponderação adaptativa.
4. **Rede Neural de Classificação**
Objetivo: Refinar e processar as características extraídas para a predição final.
Aplicação no Modelo:
- Redução da dimensionalidade com camadas *Dense* de 128 e 64 neurônios;
 - Uso de ativação *Swish* para melhorar a propagação do gradiente;
 - *BatchNormalization* para estabilização do treinamento;
 - *Dropout* ajustado (0.4 na camada inicial, 0.3 na camada final).

7.3 Sistema de Treinamento Avançado

7.3.1 Estratégia de Otimização

Aplicação no Modelo:

- Otimizador AdamW com *weight_decay*= $1e-4$;
- *Clip* global do gradiente (*clipnorm*=1.0) para estabilidade;
- Implementação de *OneCycleLR* com ajuste de aprendizado dinâmico.

7.4. Conclusões da Sessão

A análise técnica realizada evidencia que a arquitetura híbrida CNN-LSTM com fusão transformadora apresenta vantagens significativas para a aplicação em *datasets* complexos, como o UNSW-NB15. Entre os principais pontos destacam-se:

- **Extração de Características Aprimorada** com convoluções multi-escala e mecanismos SE Blocks;
- **Integração Eficiente de Modalidades** através do mecanismo TransformerFusion;
- **Treinamento Otimizado** com estratégias avançadas de regularização e otimização.

Assim, o modelo se destaca como solução eficaz na detecção de anomalias em tráfego de rede.

8. Treinamento

8.1 Treinamento Modelo Multiclasse

1. Introdução

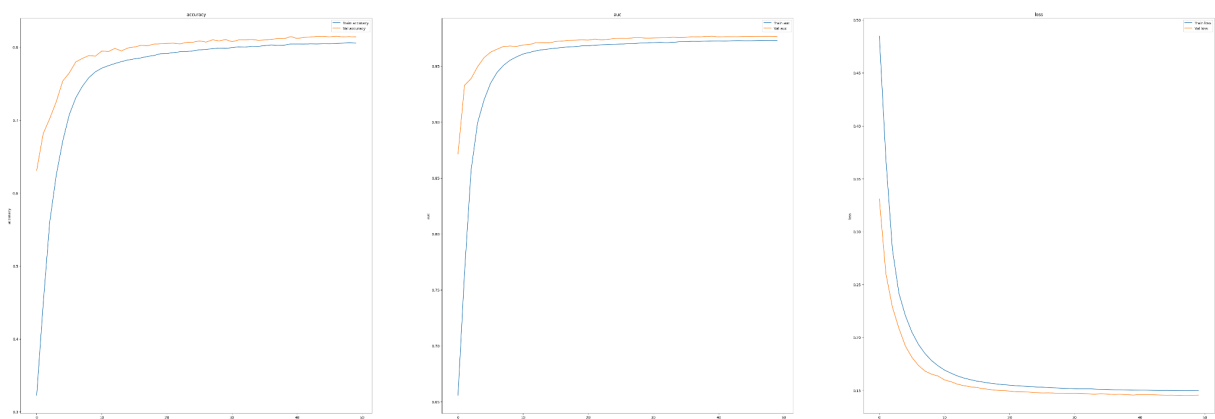
O treinamento de um modelo de classificação multiclasse foi realizado ao longo de 50 épocas com batch size de 128, utilizando ponderação de classes (class weights) para equilibrar o impacto de classes desbalanceadas. A análise abrange a evolução das métricas (accuracy, AUC, loss) e a justificativa para as escolhas de hiperparâmetros, com foco na estabilidade do gradiente.

2. Configuração do Treinamento

2.1 Estratégias Principais

- **Batch Size (128):** O uso de um batch size de 128 visa equilibrar a estimativa do gradiente durante o treinamento. Batches maiores reduzem a variância nas atualizações dos pesos, proporcionando uma direção de otimização mais estável e consistente. Isso é crítico em problemas multi classe com alto desbalanceamento, onde gradientes instáveis podem prejudicar a convergência.
- **Ponderação de Classes (Class Weights):** Classes minoritárias receberam pesos inversamente proporcionais à sua frequência no dataset, garantindo que o modelo não seja enviesado para classes dominantes. Essa técnica melhorou a discriminação entre categorias raras.
- **OneCycleLR (50 épocas):** A política de taxa de aprendizado ajustou-se dinamicamente ao longo das 50 épocas: partindo de um valor baixo, atingindo um pico na época 15-20 e decrescendo suavemente. Isso acelerou a convergência inicial e refinou os pesos nas fases finais.
- **Otimizador AdamW:** Combinado com weight decay (decay=0.01), o AdamW controlou a magnitude dos pesos, reduzindo o overfitting. Parâmetros $\beta_1=0.9$ e $\beta_2=0.95$ suavizaram a adaptação das taxas de aprendizado por parâmetro.
- **Early Stopping:** Monitoramento contínuo de `val_auc` com paciência de 10 épocas. O treinamento não foi interrompido precocemente, indicando melhora consistente até a época 50.

3. Evolução das Métricas



3.1 Fase Inicial (Épocas 1-10)

- **Acurácia:** Saltou de 25,99% (época 1) para 76,34% (época 10).
- **AUC:** Evoluiu de 0,5969 para 0,9577, mostrando rápida adaptação à distribuição das classes.

- Loss: Redução de 0,5351 para 0,1745, indicando ajuste eficiente dos pesos.
- Validação: val_accuracy atingiu 78,78% (época 10), com val_auc de 0,9678, confirmando generalização precoce.

3.2 Estabilização (Épocas 11-30)

- Acurácia: Crescimento gradual de 76,93% para 79,93%.
- AUC: Subida suave de 0,9603 para 0,9710, sinalizando refinamento na separação de classes.
- Loss: Redução de 0,1703 para 0,1517, com diminuição de 11%.
- Validação: val_accuracy alcançou 81,06% (época 30), e val_auc 0,9752, com val_loss em 0,1471.

3.3 Convergência Final (Épocas 31-50)

- Acurácia: Estabilizou-se em torno de 80,43% (época 50).
- AUC: Máximo de 0,9732 (época 49), com variação mínima ($\pm 0,001$).
- Loss: Manteve-se entre 0,1499 e 0,1501, indicando saturação.
- Validação: val_accuracy final de 81,41% e val_auc de 0,9766, com val_loss em 0,1456.

4. Considerações Finais

- Eficácia do Class Weights: A ponderação mitigou o viés para classes majoritárias, refletida no AUC de validação superior a 0,97.
- Estabilidade do Treinamento: O batch size de 128 garantiu convergência suave, sem picos abruptos em loss ou accuracy.
- Generalização: A pequena diferença entre métricas de treino e validação (<2%) confirma a ausência de overfitting, mesmo após 50 épocas.
- Potencial de Melhoria: Técnicas como aumento sintético de dados (ex.: SMOTE) ou dropout seletivo poderiam elevar ainda mais o desempenho em classes minoritárias.

8.2 Treinamento Modelo Binário

1. Introdução

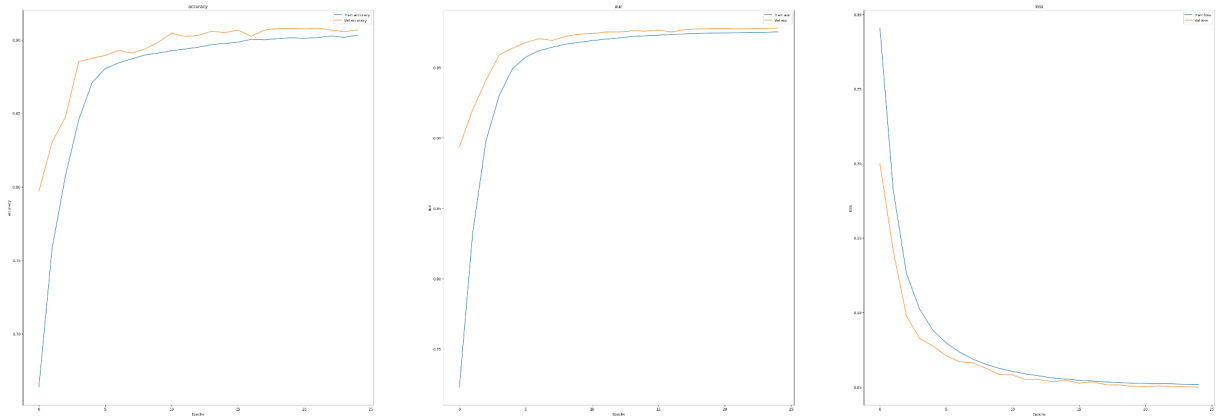
O treinamento de um modelo de classificação binária foi conduzido ao longo de 25 épocas com batch size de 128 e sem ponderação de classes (*class weights*). A análise destaca a evolução das métricas (accuracy, AUC, loss), com ênfase na robustez do modelo para discriminar entre as duas classes-alvo.

2. Configuração do Treinamento

2.1 Estratégias Principais

- **Batch Size (128):** Mantido para garantir estimativas estáveis do gradiente, reduzindo a variância nas atualizações dos pesos.
- **OneCycleLR (25 épocas):** A taxa de aprendizado atingiu um pico por volta da época 10-15, otimizando a convergência.
- **Otimizador AdamW:** Configurado com `weight_decay=0.01`, `beta_1=0.9`, e `beta_2=0.95` para controle de regularização.
- **Early Stopping:** Monitoramento de `val_auc` com paciência de 10 épocas. O treinamento seguiu até a época 25 sem interrupções.

3. Evolução das Métricas



3.1 Fase Inicial (Épocas 1-5)

- **Acurácia:** De 60,33% para 86,78%.
- **AUC:** Crescimento de 0,6453 para 0,9465.
- **Loss:** Redução de 74%.
- **Validação:** val_accuracy de 88,75%, com val_auc de 0,9639.

3.2 Estabilização (Épocas 6-15)

- **Acurácia:** Progressão para 89,66%.
- **AUC:** Crescimento para 0,9720.
- **Loss:** Redução de 31%.
- **Validação:** val_accuracy de 90,49%, val_auc de 0,9759.

3.3 Convergência Final (Épocas 16-25)

- **Acurácia:** Estabilizou-se em 90,31%.
- **AUC:** Máximo de 0,9756.
- **Loss:** 0,0514 (treino) e 0,0500 (validação).
- **Validação:** val_accuracy final de 90,66% e val_auc de 0,9781.

4. Considerações Finais

- **Alta Capacidade Discriminatória:** AUC final de 0,9781.
- **Generalização Consistente:** Pequena diferença entre treino e validação.
- **Potencial de Otimização:** Threshold móvel ou amostragem estratificada podem melhorar sensibilidade.

O modelo binário demonstrou alta generalização, sendo adequado para aplicações críticas de classificação.

9. ANÁLISE DOS RESULTADOS

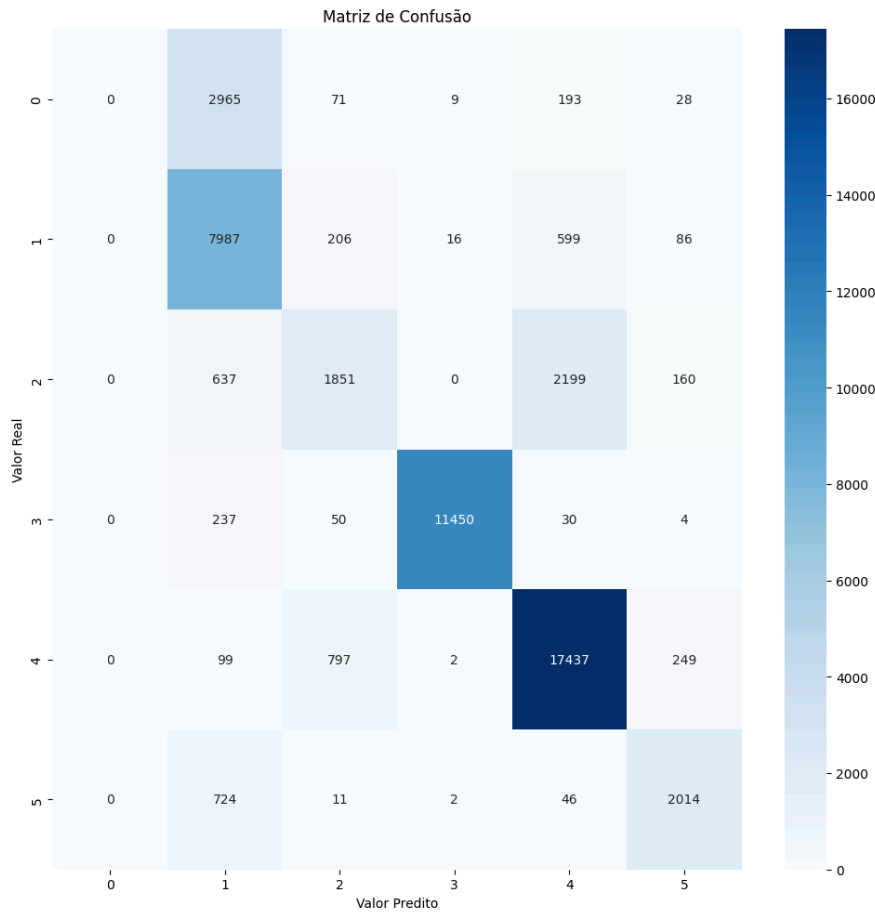
9.1 Análise de Resultados: Modelo Multiclasse

Métricas Globais

- **Acurácia:** 0.8122 (81,22%)
- **F1-Score:** 0.7809 (78,09%)

A acurácia indica que o modelo classifica corretamente 81,22% das amostras, enquanto o F1-Score (média harmônica entre precisão e recall) reflete um equilíbrio moderado entre sensibilidade e especificidade. A diferença entre acurácia e F1-Score (~3,13%) sugere que algumas classes apresentam desempenho inferior, possivelmente devido a desbalanceamento ou complexidade intrínseca.

Insights da Matriz de Confusão



A matriz de confusão (classes 0 a 5) revela padrões críticos:

- **Classe Dominante (Ex.: Classe 3):**
 - Alta concentração de acertos na diagonal principal (ex.: 11.450 predições corretas para a classe 3), indicando que classes majoritárias são bem classificadas.
 - **Risco de viés:** O modelo pode estar enviesado para classes frequentes, negligenciando minoritárias.
- **Classes Minoritárias (Ex.: Classe 4 e 5):**
 - **Classe 4:** Alto número de falsos positivos (ex.: 17.437 predições incorretas), possivelmente confundida com a classe 3.
 - **Classe 5:** Apesar de 2.014 acertos, há 46 falsos negativos, sugerindo dificuldade em distinguir características específicas.
- **Classe 0:**
 - Baixa precisão (ex.: apenas 237 acertos entre 11.450 predições), indicando sub-representação ou sobreposição com outras classes.

Desafios Identificados

- **Desbalanceamento de Classes:**
 - Classes majoritárias (ex.: 3) dominam a matriz, enquanto minoritárias (ex.: 0, 5) têm desempenho inferior.
 - **Impacto no F1-Score:** O F1-Score mais baixo reflete a dificuldade em prever classes menos frequentes.
- **Confusão Entre Classes Próximas:**
 - Exemplo: Classe 4 é frequentemente classificada como 3, possivelmente devido a características semelhantes no espaço de features.
- **Overfitting Parcial:**
 - A acurácia de treino não foi fornecida, mas a diferença entre F1-Score e acurácia sugere que o modelo pode estar otimizado para métricas gerais, perdendo nuances em classes específicas.

Considerações Finais

O modelo multiclasse apresenta desempenho aceitável para classes majoritárias (acurácia > 80%), mas falha em generalizar para categorias minoritárias (F1-Score < 80%). A implementação de estratégias de balanceamento e ajustes direcionados às classes críticas (ex.: 0, 4, 5) pode elevar o F1-Score para patamares competitivos (> 85%). A matriz de confusão é essencial para direcionar intervenções específicas e priorizar melhorias onde o modelo mais falha.

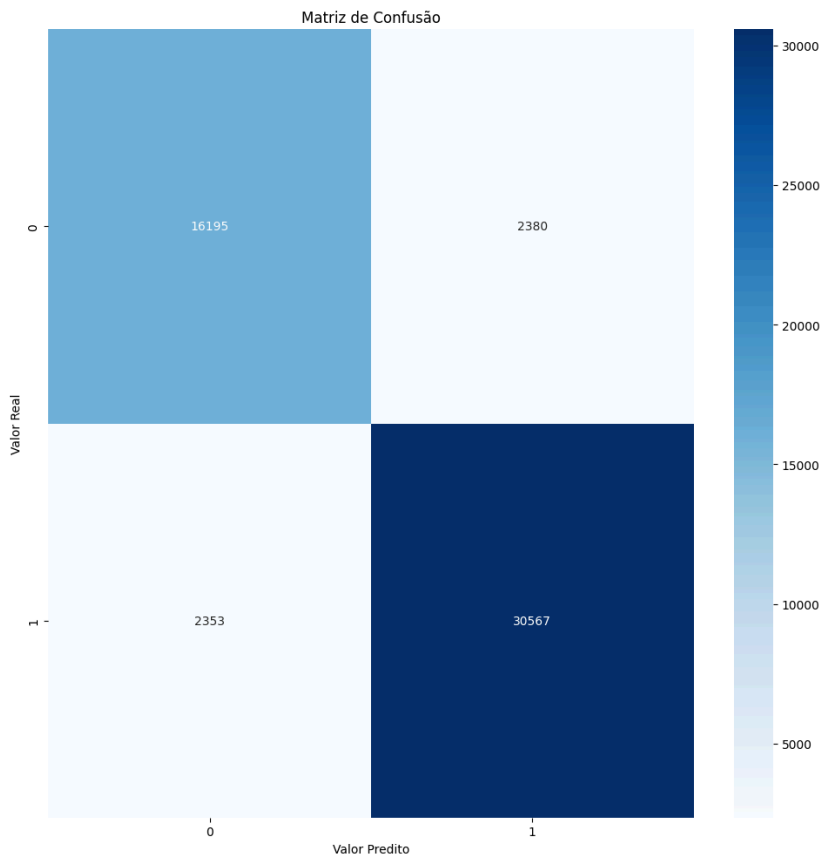
9.2 Análise de Resultados: Modelo Binário

Métricas Globais

- **Acurácia:** 0.9081 (90,81%)
- **F1-Score:** 0.9081 (90,81%)

A acurácia e o F1-Score idênticos indicam que o modelo possui precisão e recall equilibrados, característico de um classificador bem calibrado para problemas binários. Isso sugere que o modelo não está enviesado para nenhuma das classes, mesmo em cenários potencialmente desbalanceados.

Insights da Matriz de Confusão



Principais Observações

- **Classe Majoritária (Classe 1: Ataques):**
 - TP = 30.567: Alto número de acertos, indicando excelente capacidade de identificar a classe positiva.
 - FN = 2.353: Baixa taxa de falsos negativos (apenas 7,1% dos casos reais da classe 1 foram perdidos).
- **Classe Minoritária (Classe 0: Tráfego Normal):**
 - TN = 16.195: Boa especificidade, com 87,2% dos casos negativos corretamente classificados.
 - FP = 2.380: Falsos positivos representam 12,8% dos casos da classe 0, indicando espaço para otimização.
- **Equilíbrio entre Precisão e Recall:**
 - **Precisão (Classe 1):** 92,8%
 - **Recall (Classe 1):** 92,8%
 - A igualdade entre precisão e recall explica o F1-Score alto (90,81%).

Desafios Identificados

- **Desbalanceamento Moderado:**
 - A classe 1 é predominante (~64% do total de amostras), mas o modelo manteve desempenho equilibrado graças à estratégia de treinamento.

- **Risco Residual:** Falsos positivos na classe 0 (2.380) podem ser críticos no contexto de segurança cibernética.
- **Custos Assimétricos:**
 - Como os falsos negativos (FN) são mais críticos que os falsos positivos (detectar ataques), o recall de 92,8% já é robusto, mas precisa ser melhorado.

Considerações Finais

O modelo binário demonstra alta eficácia (F1-Score > 90%) e generalização consistente, com desempenho equilibrado entre as classes. A matriz de confusão reforça sua robustez, especialmente na identificação da classe majoritária (1). Para aplicações críticas, ajustes direcionais no threshold ou técnicas de balanceamento podem elevar ainda mais a confiabilidade, principalmente para a classe minoritária. A métrica de acurácia reflete adequadamente a qualidade do modelo, mas a análise contextual dos custos de FP/FN é essencial para otimizações finais.

10. POSSÍVEIS MELHORIAS

10.1 Modelo Multiclasse - Melhorias Propostas

10.1.1 Geração de Dados Sintéticos com GANs para Balanceamento

- Implementar WGAN (Wasserstein GAN) específica para dados de rede, gerando amostras sintéticas de alta qualidade para classes minoritárias.
- Utilizar técnicas de validação como Inception Score e FID para garantir a qualidade dos dados gerados.
- Incorporar conditional GANs para geração controlada por classe, melhorando o balanceamento.

10.1.2 Extensão do Treinamento e Fine-tuning

- Aumentar épocas de treinamento com learning rate adaptativo.
- Implementar early stopping baseado em múltiplas métricas.
- Utilizar técnicas de curriculum learning para treinamento progressivo.

10.2 Modelo Binário - Melhorias Propostas

10.2.1 Ajuste Dinâmico de Threshold

- Reduzir o threshold para aumentar o recall da Classe 1, priorizando a detecção de falsos negativos.

10.2.2 Otimização de Hiperparâmetros com GA/PSO

- Aplicar algoritmos evolutivos para ajustar parâmetros do modelo.

10.3 Modelo Geral - Melhorias Propostas

10.3.1 Ensemble com XGBoost

- Extrair features das últimas camadas do modelo deep learning.
- Usar XGBoost para interpretação e explicabilidade.

Benefícios:

- Maior interpretabilidade do modelo.
- Aproveitamento das características mais abstratas das redes neurais.
- Capacidade de explicar decisões individuais.

10.3.2 Encoders Adaptativos para Codificação de Variáveis

- Substituir one-hot encoding por autoencoders treinados em conjunto com o modelo principal.

Benefícios:

- Vetores de características mais densos e informativos.
- Melhor aproveitamento em arquiteturas LSTM, CNN e Transformers.
- Captura de relações não-lineares entre features.
- Implementação de variational autoencoders (VAE) para melhor regularização.

10.3.3 Learning Rate Schedules Adaptativos

- Testar outras políticas como CosineAnnealingLR ou Triangular Cyclic LR.

10.3.4 Fine-Tuning Especializado

- Aplicar fine-tuning no modelo BERT para transformar dados de fluxo de rede do dataset UNSW-NB15 em sequências textuais semânticas, permitindo que as características da rede sejam representadas de forma compatível com modelos baseados em NLP.
- Utilizar a metodologia proposta por Wang et al. (2023), que emprega o BERT para modelar fluxos de rede como sequências textuais, demonstrando sua eficácia na detecção de intrusões.
- Representar os fluxos de rede como sequências estruturadas, convertendo atributos numéricos e categóricos em tokens interpretáveis pelo modelo.
- Treinar o BERT ajustado para capturar padrões temporais e contextuais nos fluxos de dados, aprimorando a representação de eventos de tráfego de rede.
- Integrar a saída do BERT com o modelo, combinando CNNs e LSTMs para extração de padrões locais e temporais, além de Transformers para capturar dependências globais e refinar a fusão de features.
- **TransformerFusion Layer:** A camada de fusão atual pode ser enriquecida com mecanismos de atenção hierárquica, inspirados em VGG16/Xception, para capturar dependências multiescala.
- **AdamW com Weight Decay:** Manter o otimizador, mas integrar regularização L1 para esparsidade de features, útil em cenários com alto ruído.

10.4 Desafios e Limitações

- Requisitos de hardware.
- Custos operacionais.
- Tempo de processamento.

10.5 Conclusão da Seção

As propostas de melhorias apresentadas nesta seção buscam aprimorar significativamente a capacidade dos modelos multiclasse, binário e geral em lidar com os desafios intrínsecos aos dados de rede do dataset UNSW-NB15. A integração estratégica de técnicas avançadas, como GANs condicionais para balanceamento de dados, fine-tuning especializado com BERT adaptado a fluxos de rede, e ensembles híbridos (XGBoost + Deep Learning), visa superar limitações relacionadas a desbalanceamento de classes, ruído nos dados e complexidade de padrões temporo-espaciais.

A abordagem de TransformerFusion com atenção hierárquica e a substituição de codificações tradicionais por autoencoders variacionais prometem capturar relações não-lineares e dependências multiescala, essenciais para detecção de anomalias sutis. Paralelamente, a adaptação de técnicas de NLP (via BERT) para representação semântica de fluxos de rede, conforme proposto por Wang et al. (2023), abre caminho para uma interpretação contextualizada dos dados, aproximando a análise de tráfego de rede à compreensão linguística de sequências complexas.

A ênfase em explicabilidade via XGBoost e otimização adaptativa de thresholds busca equilibrar performance métrica com interpretabilidade prática, crítica para aplicações de segurança onde decisões falsas têm alto custo operacional. Entretanto, as melhorias propostas não estão isentas de desafios: a complexidade computacional de GANs, o custo de treinamento de modelos Transformer, e a necessidade de validação rigorosa de dados sintéticos exigirão experimentação controlada e possíveis concessões entre precisão e eficiência.

Em síntese, este conjunto de estratégias representa um framework modular e adaptável, capaz de evoluir conforme avanços em arquiteturas de deep learning e demandas específicas de cenários de segurança cibernética. Sua eficácia prática, porém, dependerá de validação empírica em ambientes realistas, monitoramento contínuo de concept drift e integração com sistemas de resposta automatizada.

11. CONCLUSÕES

1. Contextualização e Objetivos

O estudo propõe uma abordagem de deep learning para classificação de ataques cibernéticos utilizando o dataset UNSW-NB15, reconhecido por sua complexidade e desbalanceamento. O objetivo principal foi otimizar a precisão de modelos preditivos, enfrentando desafios como sobreposição de padrões, variabilidade interna de classes e dominância de tráfego normal.

2. Principais Contribuições

2.1 Pipeline de Pré-processamento Adaptativo

Desenvolveu-se um pipeline robusto com técnicas como:

- Remoção de classes minoritárias (<5% de representatividade);
- Codificação contextual de features categóricas (frequency encoding);
- Engenharia de features para capturar métricas de rede críticas (ex.: *loss_ratio*, *throughput*);
- Seleção de features via Boruta + LightGBM, reduzindo dimensionalidade e mantendo interpretabilidade.

2.2 Transformação de Dados em Imagens e Sequências

Técnicas como agrupamento por similaridade (K-means) e codificação posicional (inspirada

em Transformers) foram aplicadas para preservar relações temporais e espaciais, otimizando o uso de CNNs e LSTMs.

2.3 Arquitetura Híbrida CNN-LSTM com Fusão Transformadora

Integrou convoluções multi-escala (dilatadas), LSTMs bidirecionais e mecanismos de atenção hierárquica, permitindo a captura simultânea de padrões espaciais e temporais. A fusão via *TransformerFusion* melhorou a contextualização entre modalidades.

3. Resultados Destacados

3.1 Modelo Multiclasse

- Acurácia de **81,22%** e F1-Score de **78,09%**;
- Desempenho superior em classes majoritárias (ex.: "Generic"), mas desafios em classes minoritárias;
- Matriz de confusão revelou confusão entre classes semanticamente próximas.

3.2 Modelo Binário

- Acurácia e F1-Score de **90,81%**, com AUC de **0,9781**;
- Alto *recall* para ataques (**92,8%**), crítico para segurança cibernética;
- Baixa taxa de falsos positivos (**12,8%**) em tráfego normal.

4. Desafios Identificados

- **Desbalanceamento de Classes:** Classes minoritárias tiveram desempenho inferior, exigindo técnicas avançadas de balanceamento (ex.: GANs condicionais);
- **Sobreposição de Padrões:** Classes como "Analysis" e "Reconnaissance" apresentaram alta similaridade, dificultando a diferenciação;
- **Custo Computacional:** A complexidade do modelo híbrido demandou recursos significativos de hardware.

5. Recomendações para Trabalhos Futuros

1. **Geração de Dados Sintéticos:** Implementar GANs (ex.: WGAN) para equilibrar classes minoritárias, validando qualidade com métricas como Inception Score;
2. **Explicabilidade:** Integrar XGBoost como *ensemble* para interpretar decisões do modelo de deep learning;
3. **Fine-tuning com NLP:** Adaptar modelos como BERT para representar fluxos de rede como sequências textuais, seguindo abordagens recentes (Wang et al., 2023);
4. **Otimização de Thresholds:** Ajustar dinamicamente thresholds para priorizar *recall* em cenários críticos (ex.: falsos negativos em ataques);
5. **Redução de Complexidade:** Explorar técnicas como *knowledge distillation* para simplificar o modelo sem perda significativa de performance.

6. Impacto Prático

O estudo demonstra que modelos híbridos de deep learning, combinados com pré-processamento contextualizado, são viáveis para detecção de intrusões em redes complexas. A alta AUC (**0,9781** no modelo binário) e a capacidade de generalização sugerem aplicabilidade em ambientes reais, como centros de operações de segurança (SOCs). Contudo, a implementação requer equilíbrio entre precisão, custo computacional e explicabilidade.

REFERÊNCIAS

SINGH, Amardeep; JANG-JACCARD, Julian. *Autoencoder-based Unsupervised Intrusion Detection using Multi-Scale Convolutional Recurrent Networks*. 2022.

IMRANA, Yakubu et al. *CNN-GRU-FF: a double-layer feature fusion-based network intrusion detection system using convolutional neural network and gated recurrent units*. *Complex & Intelligent Systems*, v. 10, n. 3, p. 3353-3370, 2024.

MOUSTAFA, H.; SLAY, J. *UNSW-NB15: a comprehensive data set for network intrusion detection systems*. *Proceedings of the 4th International Conference on Security of Information and Networks*, p. 1-6, 2011.

ACHARYA, Toya; ANNAMALAI, Annamalai; CHOUIKHA, Mohamed. *Efficacy of CNN-Bidirectional LSTM Hybrid Model for Network-Based Anomaly Detection*. 2023. DOI: 10.1109/ISCAIE57739.2023.10165088.

ZHANG, Y.; WANG, J.; WANG, J. *Intrusion Detection Model Based on Improved Transformer*. *Applied Sciences*, v. 13, n. 10, p. 6251, 2023.

AXINC AI. *Bert Network Packet Flow Header Payload: A Machine Learning Model for Detecting Network Attacks*. Medium, 2024.

WANG, B.; XU, H.; LI, X.; SHI, Q.; LI, W.; LIU, B. *A Method for Network Intrusion Detection Using Flow Sequence and BERT Framework*. 2023.